

Integrating SDV Architecture on an AVL Robotic Truck

Gounet Noah, Wenbo Yan

June 2026

Contents

1	Introduction	3
1.1	Context and Motivation	3
1.2	Problem Statement and Objectives	3
1.3	Approach and Contributions	4
1.4	Disclaimer	4
2	Software Defined Vehicles	4
2.1	Definition and Key Characteristics	4
2.2	Opportunities	6
2.2.1	For Customers	6
2.2.2	For Manufacturers	6
2.3	Challenges	7
2.4	Market and Ecosystem Overview	8
2.5	What is VSS	9
2.6	VSS Structure	9
2.7	VSS & ROS2 : Similarities and differences	11
2.7.1	ROS2 structure	11
2.7.2	Similarities	11
2.7.3	Differences	11
2.7.4	Conclusion	12
3	Analysis of the Existing Truck Architecture	12
3.1	Analysis Scope and Method	12
3.2	Overall System Architecture	13
3.3	Docker Container Architecture	14
3.4	ROS 2 Architecture and Interfaces	14
3.5	Main Control and Data Flows	15
3.5.1	Virtual Joystick to the Wheels	15
3.5.2	Waypoints to the Wheels	16
3.5.3	Auxiliary Commands and State Feedback	16
3.5.4	Camera and Observability Paths	16
3.6	External Access Experiments	16
3.6.1	HTTP API Access	17
3.6.2	Direct ROS 2 Access	17
3.7	Current Issues and Limitations	18
4	Studied example : Eclipse SDV	19
4.1	Why Eclipse SDV	19
4.2	Eclipse Structure and main projects	19
4.3	Eclipse Leda	20
4.4	Eclipse Kuksa	20
4.5	CarSim Example	21

4.5.1	Original Example: VSS to VSS	21
4.5.2	Truck Adapted Example: ROS2 to VSS	22
4.6	ROS2 & VSS : Bridge	22
4.7	Challenges	22
4.8	Existing Solutions	23
4.9	ROS2 to VSS Bridge for AVL Truck	24
5	Conclusion	24

1 Introduction

1.1 Context and Motivation

Road vehicles are increasingly developed as distributed computing platforms rather than as collections of isolated electronic control units. Driver assistance, energy management, infotainment, diagnostics, and connectivity are implemented by software that must interact with sensors, actuators, cloud services, and external applications. In this context, a *Software-Defined Vehicle* (SDV) is a vehicle whose functions and user experience can evolve substantially through software throughout its operational life. The Eclipse SDV Working Group similarly presents the SDV as a software-centric platform whose capabilities are not fixed at the moment of production **eclipse'what'is'sdv, eclipse'sdv'charter**.

This transition creates both an architectural opportunity and an integration problem. A software-centric platform can support reusable services, remote diagnostics, continuous updates, and new vehicle applications. However, existing vehicles already contain heterogeneous middleware, hardware-specific interfaces, custom messages, and safety constraints. Replacing all of these components is generally neither realistic nor desirable. A practical transition towards an SDV architecture must therefore preserve functioning vehicle software while introducing well-defined interfaces above it.

The experimental platform studied in this project is an AVL robotic truck, referred to in the software stack as the SAVD vehicle. Its software is based on ROS 2 and distributed across a Docker Compose stack. The platform already provides manual and waypoint-based control, vehicle-state feedback, hardware interfaces, diagnostics, video streaming, a browser GUI, and an HTTP API. Nevertheless, these functions are exposed through implementation-specific ROS 2 topics, services, actions, and custom message definitions. Direct external access also depends on ROS 2/DDS discovery and on the surrounding network configuration. These properties make the truck a useful case study for investigating how an existing robotic vehicle can participate in an SDV-oriented software environment.

1.2 Problem Statement and Objectives

The general problem addressed by this project is how to connect an existing ROS 2-based vehicle platform to an SDV environment without replacing its internal control architecture. The integration must make selected vehicle information accessible through a more stable and semantically explicit interface while respecting the separation between remote applications, vehicle software, and physical control.

The project pursues the following objectives:

- O1:** establish the conceptual background of software-defined vehicles, including their expected benefits, limitations, and current ecosystem;
- O2:** analyse the running truck architecture, including its Docker containers, ROS 2 interfaces, and principal control and data flows;
- O3:** study the role of VSS as a common vehicle-data model and examine the available open-source SDV components;
- O4:** investigate a bridge between the existing ROS 2 platform and a VSS-based data environment; and
- O5:** assess practical accessibility and deployment constraints through reproducible, non-destructive experiments.

The scope is deliberately limited. The work does not redesign the low-level controller, replace ROS 2, or claim production-level functional-safety certification. Instead, it concentrates on understanding the existing platform and introducing an integration layer around selected vehicle data and services. Physical-control tests are outside the architecture-inspection scope unless they are performed under a separate, explicitly secured test procedure.

1.3 Approach and Contributions

The project combines background research, architecture analysis, prototype-oriented integration work, and experimental validation. First, standards and open-source initiatives were reviewed to identify the characteristics of an SDV and the role of common vehicle interfaces. Second, the live truck platform was inspected in read-only mode. The inspection covered the Docker Compose configuration, running containers, mounted source trees, launch files, ROS 2 graph, custom interfaces, GUI implementation, and HTTP API. Third, remote access through the existing HTTP API was compared with direct ROS 2 discovery in several network environments. Finally, the resulting architecture model was used to guide the ROS 2–VSS integration described in the later sections.

The principal contributions of the work are:

- a structured overview of SDV concepts, opportunities, challenges, and ecosystem initiatives;
- a runtime-grounded decomposition of the truck into web, control, vehicle-abstraction, hardware-interface, sensor, media, and observability layers;
- a mapping of the main ROS 2 topics, services, actions, and end-to-end control and data flows;
- a comparison of HTTP-based remote access and direct ROS 2 access from a macOS Docker environment over a VPN, from the truck itself, and from a Linux host on the same LAN; and
- a documented set of current platform constraints and limitations relevant to subsequent bridge integration and testing.

Section 2 introduces SDV concepts and the surrounding ecosystem such as VSS and its structure. Section 3 analyses the existing truck. The following section study the Eclipse SDV ecosystem, and present the selected implementation examples and describe the bridge concept.

1.4 Disclaimer

During this project AI has been used at different steps. To help us analyses the truck architecture and write the related Readme.md file. To help us fix some issues during the set up of the CarSim examples, and in the writing of this report to help us translate to english and correct our work.

2 Software Defined Vehicles

2.1 Definition and Key Characteristics

There is no single universally binding definition of a software-defined vehicle. The term is used by manufacturers, suppliers, standardisation bodies, and open-source communities with slightly different emphases. A useful engineering definition is the following:

An SDV is a vehicle whose functions, behaviour, and user-facing capabilities are primarily implemented and continuously evolved through software running on a reusable computing and communication platform.

This definition goes beyond the fact that a modern vehicle contains software. Conventional vehicles have contained millions of lines of code for decades, but their software has often been organised around hardware-specific electronic control units and fixed at production time. In an SDV, software becomes the main mechanism for adding, changing, deploying, and operating vehicle functions across the vehicle lifecycle. Eclipse SDV describes this transition as the movement towards a modular and extensible vehicle software platform that supports in-vehicle and around-vehicle application development **eclipse·what·is·sdv**, **eclipse·sdv·charter**.

Five characteristics are particularly important.

Software–hardware decoupling. Applications should depend on stable services and interfaces rather than directly on a particular sensor, actuator, or controller implementation. Hardware abstraction does not eliminate hardware constraints; it prevents them from propagating unnecessarily into every application.

Consolidated and service-oriented architecture. Vehicle functions are increasingly moved from many function-specific controllers towards domain, zonal, or central computers. Services and well-defined APIs allow different applications to reuse capabilities such as positioning, diagnostics, vehicle state, and actuation. AUTOSAR Adaptive is one industry response to this need for service-oriented applications on high-performance computing platforms `autosar'adaptive'platform`.

Continuous software lifecycle. The vehicle is treated as a maintained software product after delivery. Remote diagnostics, telemetry, controlled over-the-air updates, version management, and rollback mechanisms become part of normal fleet operation. Software deployment is therefore linked to update governance and regulation, including UNECE Regulation No. 156 on software-update management systems `unece'r156`.

Common data semantics. A vehicle application must understand not only how to receive a value, but also what that value represents, its unit, its valid range, and its relationship to other signals. Common data models reduce the amount of vehicle-specific mapping required by applications. COVESA’s Vehicle Signal Specification is one example of an open, hierarchical model for describing vehicle data `covesa'vss`.

Vehicle–edge–cloud continuity. Development, testing, deployment, and monitoring extend beyond the in-vehicle computer. Cloud and edge tooling can support simulation, fleet management, data analysis, and application delivery. The SOAFEE initiative focuses on bringing cloud-native development approaches to automotive edge platforms while retaining the constraints of automotive systems `soafee'overview`.

Table 1: Simplified comparison between hardware-centric and software-defined vehicle development.

Aspect	Hardware-centric approach	Software-defined approach
Function ownership	Closely coupled to a specific ECU or supplier component	Implemented as reusable software services on shared platforms
Vehicle evolution	Major changes normally tied to a new vehicle programme	Selected functions can evolve after production through managed updates
Interfaces	Hardware- and project-specific signals	Stable APIs and explicit data semantics
Computing layout	Large number of distributed function-specific ECUs	Greater use of domain, zonal, and central computing
Development cycle	Strongly coupled hardware and software releases	Continuous integration, virtual testing, and staged deployment
External integration	Added separately for each function	Designed around controlled vehicle, edge, and cloud interfaces

An SDV should not be confused with a merely *connected vehicle*. Connectivity allows data exchange with external systems, but it does not by itself imply modular software, stable abstraction layers, or continuous functional evolution. Similarly, an SDV is not necessarily an autonomous vehicle. Automated-driving functions can benefit from an SDV platform, but the defining property is the software-centric lifecycle and architecture, not a particular driving-automation level.

2.2 Opportunities

The value of an SDV comes from separating the useful lifetime of vehicle functions from a single hardware release. The same platform can offer different benefits to vehicle users and to the organisations that design, manufacture, and operate vehicles.

2.2.1 For Customers

Post-sale improvement. Software updates can correct defects, improve energy management, refine user interfaces, and add compatible features without a workshop visit. The important benefit is not simply remote installation; it is the possibility of maintaining a vehicle as an evolving product. A safe implementation requires authenticated delivery, compatibility checks, staged rollout, and rollback.

Personalisation and feature continuity. Profiles and software-configured features can adapt displays, comfort functions, charging strategies, or driver-assistance settings to different users. When interfaces remain stable across models, customer-facing applications can also be reused more consistently.

Faster diagnosis and maintenance. Structured telemetry and remote diagnostics can reveal faults before a physical inspection. Fleet operators can identify recurring failures, schedule maintenance more effectively, and distinguish software problems from hardware faults. For individual users, this can reduce downtime and improve the clarity of service interventions.

A broader application ecosystem. Stable, permission-controlled vehicle services allow new applications to use vehicle data without knowing every internal ECU or bus signal. This can enable navigation, accessibility, energy, insurance, fleet, and infotainment services. The benefit depends on strong privacy controls and on ensuring that third-party applications cannot bypass safety boundaries.

Longer functional relevance. A vehicle may remain useful for many years. If software components can be maintained independently of the original infotainment or compute stack, selected functions can remain compatible with changing services and security requirements. This opportunity is conditional: long-term support must be planned, funded, and technically feasible on the installed hardware.

2.2.2 For Manufacturers

Reuse across models and programmes. A common software platform allows manufacturers to reuse diagnostics, communication, deployment, and application services across several vehicle lines. Reuse can reduce duplicated integration effort and let product teams focus on differentiating functions.

Shorter and more observable development cycles. Virtualised development environments, automated tests, and software-defined interfaces allow parts of the system to be developed before target hardware is available. Continuous integration can expose interface regressions earlier than vehicle-level testing alone. Initiatives such as SOAFEE and Eclipse SDV explicitly promote reusable, cloud-informed automotive development practices [soafee'overview](#), [eclipse'sdv'charter](#).

Fleet-level operation. Manufacturers can monitor deployed software versions, diagnose incidents, roll out fixes in stages, and collect evidence about software quality across a fleet. This changes the manufacturer's responsibility from delivering a finished product once to operating a software platform over many years.

New services and business models. Optional software functions, fleet services, predictive maintenance, and data-enabled services may create revenue after the initial vehicle sale. However, commercial opportunity does not justify locking basic functionality behind unstable subscriptions or collecting unnecessary personal data. Sustainable services require clear customer value, transparent terms, and regulatory compliance.

A more modular supplier relationship. Explicit interfaces can reduce dependence on opaque, vertically integrated components. Manufacturers and suppliers can collaborate around common platforms while retaining differentiated implementations. Open-source foundations can host non-differentiating infrastructure—for example communication, data models, development tooling, and reference runtimes—that would otherwise be repeatedly implemented by each organisation.

2.3 Challenges

The SDV transition does not remove automotive constraints. It introduces modern software practices into a domain where failures can affect physical safety and where products may remain in service for more than a decade.

Architectural complexity and legacy integration. New service-oriented applications must co-exist with existing buses, controllers, diagnostic protocols, and supplier software. A central platform may simplify application development while increasing the criticality of shared compute, middleware, and networking. Migration therefore normally requires adapters and abstraction layers instead of an immediate replacement of the complete vehicle stack.

Real-time and resource constraints. Cloud-native concepts cannot be copied into a vehicle without adaptation. Vehicle functions have bounded latency, startup, memory, power, thermal, and availability requirements. Container isolation and dynamic deployment are useful, but safety-critical control loops still require deterministic behaviour and careful resource allocation. The computing architecture must distinguish convenience applications, quality-managed functions, and safety-relevant control.

Functional safety and update assurance. An update may change a previously validated function or an interaction between functions. Safety analysis must therefore cover software variants, dependencies, degraded modes, and rollback. ISO 26262 provides the principal functional-safety framework for road-vehicle electrical and electronic systems, while update processes are addressed by UNECE Regulation No. 156 **iso26262**, **unece'r156**. Continuous delivery in this domain means continuous evidence and controlled release, not unrestricted deployment.

Cybersecurity and privacy. Connectivity, remote diagnostics, application APIs, and update services increase the attack surface. Threats include unauthorised control, malicious updates, credential theft, denial of service, and extraction of location or behavioural data. ISO/SAE 21434 and UNECE Regulation No. 155 establish lifecycle-oriented cybersecurity expectations for road vehicles **iso21434**, **unece'r155**. An SDV platform consequently needs secure boot, signed updates, identity and access management, network segmentation, logging, vulnerability response, and data minimisation.

Interoperability and semantic consistency. Two components can use compatible transports and still disagree about signal names, units, coordinate frames, validity, or timing. A common API must define semantics, not only message delivery. Versioning is equally important because applications, data models, and vehicle platforms evolve at different rates.

Organisational transformation and long-term support. SDV development spans embedded engineering, cloud platforms, cybersecurity, safety, data governance, and vehicle operations. Team structures and supplier contracts built around isolated ECUs can conflict with end-to-end software ownership. In addition, cryptographic algorithms, operating systems, and cloud dependencies can become obsolete before the vehicle is retired. Manufacturers must therefore plan platform governance, maintenance responsibilities, update infrastructure, and service continuity for the complete support period.

2.4 Market and Ecosystem Overview

The SDV market is better understood as an evolving ecosystem than as a single product category. Vehicle manufacturers increasingly announce central-compute platforms, common operating environments, and software-update capabilities, while suppliers offer middleware, high-performance computers, cloud platforms, and engineering tools. Adoption remains uneven: connected infotainment and remote updates are already common, but cross-domain application platforms and broadly portable vehicle applications are still developing.

Market-size forecasts should be interpreted cautiously because different reports count different combinations of vehicle software, cloud services, semiconductors, engineering services, and connected features. This report therefore focuses on observable ecosystem roles rather than presenting one forecast value.

Table 2: Selected organisations and initiatives in the SDV ecosystem.

Initiative	Primary role	Relevance to SDVs
AUTOSAR Adaptive	Automotive application platform	Service-oriented runtime and standardised interfaces for high-performance vehicle computers autosar`adaptive`platform
COVESA	Industry alliance and open specifications	Common vehicle-data semantics and connected-vehicle specifications, including VSS covesa`about</b> , <b>covesa`vss
Eclipse SDV	Open-source working group	Vendor-neutral projects for in-vehicle, edge, cloud, development, and fleet software eclipse`sdv`charter , eclipse`sdv`landscape
SOAFEE	Reference architecture and industry community	Cloud-native-inspired development and deployment for automotive edge computing soafee`overview
UNECE WP.29	Regulatory framework	Cybersecurity and software-update management requirements through Regulations No. 155 and No. 156 unece`r155</b> , <b>unece`r156
ISO	International standards	Lifecycle processes for functional safety and automotive cybersecurity through ISO 26262 and ISO/SAE 21434 iso26262 , iso21434

These initiatives are complementary rather than interchangeable. AUTOSAR defines automotive software-platform standards; COVESA works on common vehicle-data and connected-vehicle specifications; Eclipse SDV hosts implementable open-source projects; SOAFEE concentrates on development and deployment patterns for automotive edge platforms; and ISO and UNECE define safety, cybersecurity, and update constraints. A production vehicle may use elements from several of them.

The Eclipse SDV Working Group is especially relevant to this project because it provides open-source components that can be studied and deployed without access to a proprietary manufacturer platform. Its stated mission is to support a scalable, modular, extensible, industry-ready open vehicle software platform, and its landscape covers projects for in-vehicle and cloud-side use **eclipse`sdv`charter**, **eclipse`sdv`landscape**. This makes the ecosystem suitable for a research prototype that must coexist with an existing ROS 2 vehicle.

The ecosystem overview also clarifies why analysing the truck before selecting an integration mechanism is necessary. Standards and frameworks cannot determine which existing vehicle interfaces are trustworthy, which data are available, or how control is arbitrated. Those facts depend on the deployed platform. Section 3 therefore moves from the general SDV context to a runtime-grounded analysis of the AVL robotic truck.

2.5 What is VSS

Vehicle Signal Specification (VSS) is an open source and standardized hierarchical tree format developed since 2016 by COVESA for automobile telemetry variables. As mentioned in part , one of the most important challenge that SDVs are encountering is the system compatibility fragmentation. The vehicles have to be able to communicate to each other and understand each other. However, in front of the number of manufacturers that are all defining their own intern standards, it is necessary to find a common standard for V2V and V2X communication. It is with this goal in mind that COVESA started to develop VSS. The goal is to create a standard hierarchical tree format with the main objective of being easy to understand and to use, and to cover a large amount of contexts to be suitable to all the automotive industry. It has been developed to allow COVESA to enable vehicle data interoperability, reduce fragmentation and facilitate collaboration between the different industries of the Automotive world (EV charging, safety, fleet operation maintenance etc). It aims to accelerate the SDV ecosystem growth and to create new business opportunities. (Some elements of this part have been directly rephrase from COVESA WEBSITE)

2.6 VSS Structure

VSS has been created following some key principles. According to COVESA, it has been developed to be :

- “Simple, flexible, and easily understandable by machine and humans”
- “Developer friendly and extensible, with the support industry-supported tooling .
- “Represent the vehicle domain in multiple contexts/perspectives of the mobility ecosystem”.
- “Support a robust variety of vehicles including consumer and commercial vehicles.’
- “Protocol and technology agnostic.”

VSS has been developed with the objective of being widely used. It is designed to be easy to use and understood by every actor of the industry to avoid complex and expensive formation/adaptation for the companies. To cover as many sectors of the automotive industry and not only the specific need of the car manufacturers, in order to be used by everyone, for largest range possible of V2V and V2X communications. Finally it has been created to be able to evolve at least as fast as the industry, but also to be source of evolution for the industry. It was not only created to solve a problem, but also to make sure that it will be largely adopted.

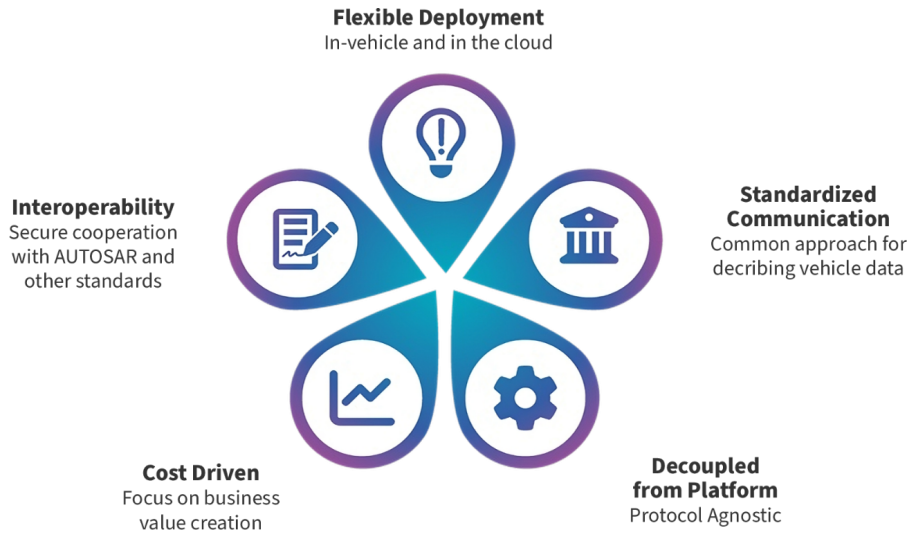


Figure 1: key design principles of VSS

VSS is designed as a tree. It started from a root the “vehicle” and is composed of Nodes that are parts of the car such as the Chassis, the cabin etc, and finally the leaves that are the sensors and the actuators. Every part is labeled with a human readable name ‘Vehicle’, “Chassis”, “Seat”, “Pressure”, “Islocked” etc. The goal is to make it both human and machine readable. All of the leaves elements are defined in a file that can be of different type depending of the usage etc (csv, graphql, json, fidl, proto, yaml, etc) with a name composed of the chain from the root to the leave for example Vehicle.Cabin.Seat.IsBelted, a Datatype (boolean, 8Int, etc) a type (sensor etc) and a Description of what it’s represent.

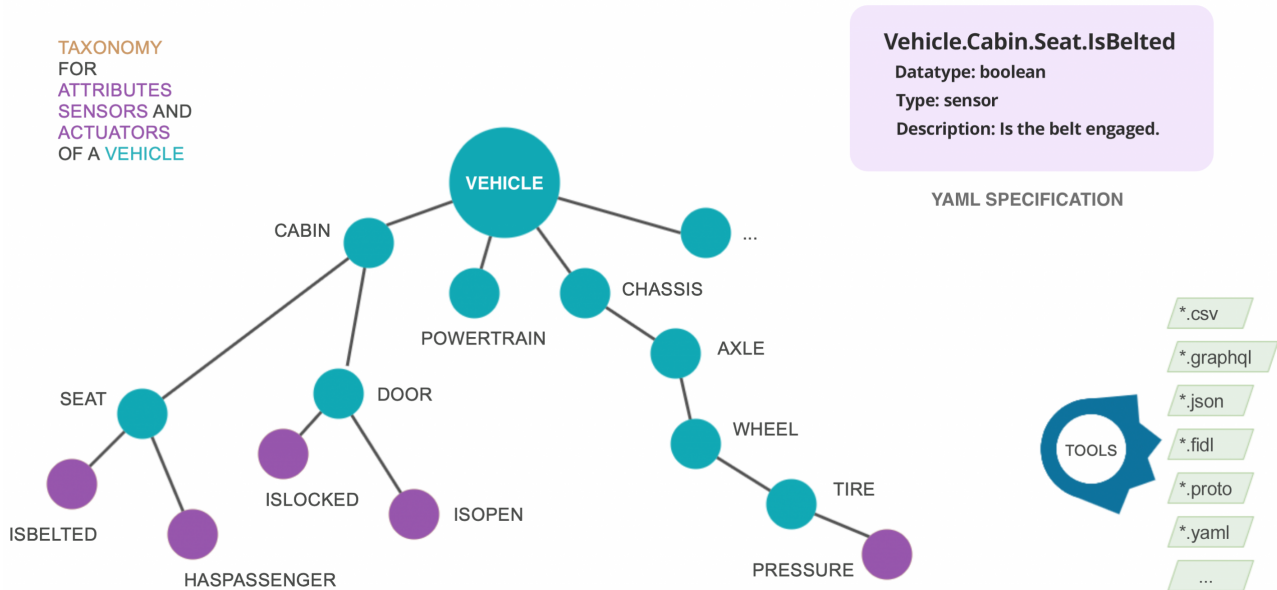


Figure 2: tree architecture of VSS

The structure is simple and well organized. It allows the users to understand easily what is the received data (Datatype and description). From what type of elements it is coming from and where in the system it is, we just have to follow the tree path contain in the name of the element to understand the general organization of the car. Moreover it make the different element easy to discover. If we are looking for a special element in the car, we can go down the tree until reaching the list of sensors and actuators and find the desired one, avoiding to go through a list of thousands of sensors. VSS is also machine friendly, as the data are structured, it make it easier to create the programs dealing with it, to map the different data from the sdv ecosystem to VSS. As every vehicle is different and contain different sensors and so on that can be unique to a model, VSS furnish a

website : <https://vss.digital.auto/> where a user can register or publish a vehicle tree structure allowing the other users to access the structure of the vehicle and the specific elements of the car.

2.7 VSS & ROS2 : Similarities and differences

Now that we explored the structure of VSS, we want to compare it with ROS2 used on our truck to understand how to map them together and make the truck able to communicate with other vehicles, using VSS standard. First of all it is important to understand that ROS2 and VSS are two totally different but complementary communication standards that solve different problems. ROS2 is a communication middleware, its goal is to move data between running processes. VSS is a data model, it defines what a piece of vehicle data is called and what type it has, but doesn't say anything about how it travels over a wire.

2.7.1 ROS2 structure

ROS2 is built around "node". A node is an independent process doing one job (a camera driver, a planner, a controller), and that talk to other nodes through:

- **Topics** : asynchronous publish/subscribe streams (e.g. a camera node publishes `image_raw`, a perception node subscribes to it)
- **Services** : synchronous request/reply calls for one-off operations
- **Actions** : long-running goal-oriented tasks with feedback and cancellation (e.g. "navigate to this point")
- **Parameters** : runtime-configurable values per node

ROS2 is built on Data Distribution Service (DDS) via an abstraction layer called 'rmw'. DDS handles peer-to-peer discovery, serialization, and Quality-of-Service policies.

2.7.2 Similarities

- Both are hierarchical and standardized. ROS2 namespaces nodes and topics into trees (for example, `/robot1/camera/image_raw`). VSS organizes signals into a tree (for example, `Vehicle.Cabin.Door`). Both provide an organized, human- and machine-readable scheme instead of ad-hoc strings.
- Both are open source, ROS2 under Open Robotics/the ROS Foundation, VSS under COVESA, with public repos
- Both define typed schemas. ROS2 uses `.msg`, `.srv`, and `.action` IDL files; VSS uses `.vspec` files with datatype, unit, and range information for each signal.

2.7.3 Differences

What it is:

- **ROS2**: A middleware/communication framework
- **VSS**: A data model / signal catalog (no transport)

Domain:

- **ROS2**: General robotics (any sensors, actuators, control loops)
- **VSS**: Automotive only

Core unit:

- **ROS2**: Node (a running process)

- **VSS:** Signal (a named, typed value in a tree)

Interaction patterns:

- **ROS2:** Topics (pub/sub), services (request/reply), actions (goal/feedback)
- **VSS:** None defined. Access is guaranteed by separate tools such as [Eclipse Kuksa](#)

Transport:

- **ROS2:** DDS, peer-to-peer discovery, configurable QoS
- **VSS:** Not specified by VSS itself, depend on the databroker

Schema scope:

- **ROS2:** App-specific — user defines its own message types per project
- **VSS:** One shared and maintained catalog that can be extended. Everyone draws from.

Topology:

- **ROS2:** Decentralized graph of independent peers
- **VSS:** Centralized tree, typically served by one broker process

Real-time fit:

- **ROS2:** Built for control loops, low-latency sensor/actuator data
- **VSS:** Built for state representation and interoperability, not hard real-time control

2.7.4 Conclusion

They're increasingly used together. Autonomous-vehicle stacks often run ROS2 internally for perception/planning/control, and expose standardized state to apps and cloud services through VSS via a broker. For example one COVESA reference blueprint shows a Kuksa Zenoh provider forwarding messages between the API of the Eclipse Kuksa Databroker and topics on a Zenoh network, where a VSS signal like `Vehicle.Body.Horn.IsActive` is mapped directly to a pub/sub topic.

3 Analysis of the Existing Truck Architecture

3.1 Analysis Scope and Method

The purpose of this analysis was to establish an evidence-based model of the software currently deployed on the AVL robotic truck. This step was required before designing any external integration: container names or source-level assumptions alone do not show which components are active, how ROS 2 names are remapped at launch time, or which path is authorised to reach the hardware.

The primary inspection was performed on 2 June 2026. At that time, the vehicle host was running Ubuntu 22.04.5 LTS on an NVIDIA Jetson `armv8` platform with Docker 20.10.17. The inspection was deliberately read-only. No remote file was modified, no container was restarted, no recovery script was executed, and no command capable of moving the vehicle was sent `yan'saved'inspection`.

The analysis combined the evidence sources listed in [table 3](#). Runtime observations were cross-checked with launch files and source code because each source alone can be misleading. For example, a healthy Docker container only proves that its health check succeeded; it does not prove that the associated sensor is delivering valid data. Similarly, a relative topic name in C++ or Python can become a different fully-qualified name after ROS 2 namespace and launch-file remapping.

Table 3: Evidence used for the truck architecture analysis.

Evidence source	Information obtained
Docker Compose configuration	Declared services, images, commands, networks, volumes, devices, environment variables, and health checks
Live Docker state	Running and exited containers, effective commands, mounts, logs, and process state
ROS 2 graph	Nodes, topics, message types, services, actions, namespaces, and runtime remappings
Source and launch files	Component ownership, conversions, state machines, API routes, and launch-time configuration
GUI and OpenAPI descriptions	External operations and the mapping from browser actions to API calls
Read-only API requests	Availability of selected mode, parameter, and diagnostic information
Remote-access experiments	Behaviour of HTTP and direct ROS 2 discovery in different host and network environments

The remote-access comparison was conducted on 3 June 2026. It used only read-only API requests and ROS 2 graph discovery. The addresses shown in the original experiment notes are deployment-specific and are replaced by `<TRUCK_IP>` in the report. The complete commands and observations are retained in the internal project documentation `yan'remote'access'test`.

3.2 Overall System Architecture

The truck is not controlled by one monolithic application. It is a distributed ROS 2 system whose components are packaged into Docker containers. At the highest level, a browser request enters through a web API, is converted into a ROS 2 operation, passes through mode management and command arbitration, reaches a vehicle-abstraction node, and is finally translated into commands for the VESC motor controller or the ESP32-based auxiliary controller.

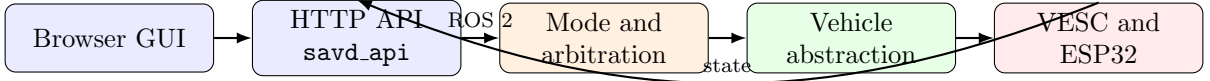


Figure 3: Simplified command and feedback path of the SAVD truck.

The most important architectural boundary lies between command producers and the vehicle abstraction. Manual control and waypoint operation can both produce drive commands, but the vehicle does not subscribe directly to every producer. Instead, `savd.sysmode` owns the mode state machine and `savd.syscontrol` selects the single drive source allowed in the current mode. The selected command is republished on `/savd.syscontrol/drive_cmds`, which is consumed by `savd.vehicle`. Consequently, the existence of a command on an upstream topic does not imply that it can move the vehicle.

The overall safety shape can be summarised as follows:

```

multiple possible command producers
-> mode state machine
-> one command arbiter
-> vehicle abstraction
-> hardware interfaces

```

State flows in the opposite direction. VESC measurements and ESP32 state are converted into ROS 2 messages by the hardware-interface containers. `savd.vehicle` republishes higher-level battery, odometry, and vehicle-parameter information. The API subscribes to this state and exposes selected values as JSON to the GUI and remote clients.

3.3 Docker Container Architecture

The active Compose project was `savd_docker`. Its effective configuration combined a base Compose file with ZED-camera and health-check overrides. Eighteen services were defined; 17 containers were running, while `savd_jetson_stats` was exited. A second GUI container existed in the Docker inventory in the `Created` state, but it belonged to a different Compose project and was not part of the active runtime.

Rather than treating the services as unrelated processes, they can be grouped into the functional layers shown in [table 4](#).

Table 4: Functional layers of the deployed Docker stack.

Layer	Containers	Responsibility
Web and API	<code>savd_gui</code> , <code>savd_api</code>	Browser interface, REST routes, and conversion between HTTP and ROS 2
Mode and arbitration	<code>savd_sysmode</code> , <code>savd_syscontrol</code>	Mode state machine and selection of the authorised drive-command source
Command producers	<code>savd_manop</code> , <code>savd_wpo</code> , <code>savd_teleop</code>	Virtual or physical joystick processing and waypoint-following commands
Vehicle abstraction	<code>savd_vehicle</code>	Conversion from selected motion commands to Ackermann commands, auxiliary commands, and higher-level state
Hardware interfaces	<code>vesc_ackermann</code> , <code>vesc_driver</code> , <code>savd_micro_ros_agent</code>	VESC command conversion, VESC serial communication, and ESP32 micro-ROS serial transport
Sensors and media	<code>savd_ublox</code> , <code>savd_zed_gstreamer</code> , <code>savd_mediamtx</code>	GPS or static transforms, ZED RTSP production, and RTSP/HLS/WebRTC relay
Observability	<code>savd_diagnostics</code> , <code>savd_foxglove_bridge</code> , <code>savd_foxglove_studio</code> , <code>savd_jetson_stats</code>	Diagnostic aggregation, ROS 2 visualisation, and host statistics

Containerisation gives each component a reproducible runtime and makes dependencies visible, but it also creates operational subtleties. Hardware devices and serial ports must be mounted into the correct container, ROS 2 discovery must traverse the effective Docker network, and container health checks can only test what they were designed to observe. During the inspection, for example, `savd_teleop` could be marked healthy while waiting for a Logitech F710 process even though no joystick device was available. The `savd_ublox` container was also healthy while its active launch file started only static transform publishers and not the actual GPS node.

3.4 ROS 2 Architecture and Interfaces

ROS 2 provides the internal communication model of the truck. Topics are used for streams such as commands, state, odometry, and diagnostics; services are used for request/response operations such as mode or vehicle-parameter changes; and an action is used for waypoint execution because it requires a long-running goal, feedback, and cancellation.

[Table 5](#) lists representative interfaces that define the main architecture. The table uses runtime names rather than source-level relative names.

Table 5: Representative ROS 2 interfaces in the running truck.

Runtime interface	Type	Purpose
/savg_sysmode/mode	std_msgs/msg/String	Current vehicle mode
/savg_manop/joy_cmds_2	sensor_msgs/msg/Joy	Virtual joystick input from the API
/savg_manop/drive_cmds	geometry_msgs/msg/TwistStamped	Manual-operation motion command
/savg_wpo/drive_cmds	geometry_msgs/msg/TwistStamped	Waypoint-operation motion command
/savg_syscontrol/drive_cmds	geometry_msgs/msg/TwistStamped	Command selected by the mode-dependent arbiter
/ackermann_cmd	ackermann_msgs/msg/AckermannDriveStamped	Vehicle-level steering and speed command
/commands/motor/speed	std_msgs/msg/Float64	VESC motor-speed command
/commands/servo/position	std_msgs/msg/Float64	Steering-servo command
/savg_micro_ros/cmd	savg_interfaces/msg/SAVDCCommand	Auxiliary command sent to the ESP32
/savg_micro_ros/state	savg_interfaces/msg/SAVDState	State received from the ESP32
/savg_vehicle/battery_state	sensor_msgs/msg/BatteryState	Higher-level battery state
/savg_vehicle/odom	nav_msgs/msg/Odometry	Vehicle odometry exposed upstream
/diagnostics_agg	diagnostic_msgs/msg/DiagnosticArray	Aggregated subsystem diagnostics
/savg_wpo/waypoints	savg_interfaces/action/Waypoints	Waypoint goal, feedback, result, and cancellation

The truck also defines custom interfaces. **SAVDCCommand** includes fan, gear, front and rear differential, VESC-on, shutdown, and error-acknowledgement fields. **SAVDState** returns the corresponding low-level state together with bumper, mode, shutdown, and fault information. The **Waypoints** action carries a **nav_msgs/Path** goal and returns progress feedback. These definitions are important for local ROS 2 applications because a remote subscriber needs the same interface package to decode custom messages.

Many nodes create relative names such as **drive_cmds**, **mode**, or **set_mode**. Launch files apply namespaces and explicit remappings to produce the runtime names. For instance, the **drive_cmds** subscription created by **savg_vehicle** is remapped to **/savg_syscontrol/drive_cmds**. Reading the source without the launch configuration would therefore suggest an incomplete or incorrect data path. The analysis consistently uses the effective runtime graph as the primary reference and source code to explain behaviour.

3.5 Main Control and Data Flows

3.5.1 Virtual Joystick to the Wheels

The browser’s virtual joystick does not communicate with the motor controller directly. The end-to-end path is:

```

Browser GUI
-> PUT /manual/send_joy_cmds
-> savd_api
-> /savg_manop/joy_cmds_2
-> savd_manop
-> /savg_manop/drive_cmds
-> savd_syscontrol (mode-dependent forwarding)
-> /savg_syscontrol/drive_cmds
-> savd_vehicle
-> /ackermann_cmd
-> vesc_ackermann
-> /commands/motor/speed + /commands/servo/position
-> vesc_driver
-> VESC hardware

```

This path contains two distinct conversions. **savg_manop** converts joystick axes to a velocity and curvature command, while **vesc_ackermann** converts the Ackermann representation to motor ERPM and steering-servo position. Between them, **savg_syscontrol** enforces which producer is active. A valid manual command is therefore ignored unless the current mode metadata selects the manual-operation topic.

The physical joystick follows the same downstream path but enters through `savd_teleop` and `/savd_manop/joy_cmds`. The manual-operation implementation gives this physical input priority over the virtual joystick while it is active.

3.5.2 Waypoints to the Wheels

Waypoint operation is modelled as a ROS 2 action rather than a direct velocity request:

```
Browser map
-> PUT /wpo/send_waypoints
-> savd_api action client
-> /savd_wpo/waypoints
-> waypoint action server
-> path segment + TF
-> pure-pursuit node
-> curvature and current pose
-> /savd_wpo/drive_cmds
-> savd_syscontrol
-> same vehicle and VESC chain as manual operation
```

The path-following controller depends on a valid transform chain. If the required transform between the global coordinate frame and `base_link` is unavailable, the pure-pursuit node cannot derive a reliable current pose and curvature. As with manual control, waypoint commands reach the vehicle only in a mode that selects the waypoint-operation drive topic.

3.5.3 Auxiliary Commands and State Feedback

Gear, differential lock, fan speed, and VESC power are not encoded in the motion-command topic. They enter through separate HTTP routes and ROS 2 services owned by `savd_vehicle`. The vehicle node stores the requested state and periodically publishes a custom `SAVDCommand` message on `/savd_micro_ros/cmd`. The micro-ROS agent transports that message over serial to the ESP32 firmware.

Feedback follows several paths. VESC measurements are published on `/sensors/core`; odometry is produced by the VESC/Ackermann subsystem; and ESP32 state arrives on `/savd_micro_ros/state`. `savd_vehicle` converts or republishes this information as battery state, odometry, and consolidated vehicle parameters. The API subscribes to those topics, and the GUI retrieves the latest values through REST endpoints.

3.5.4 Camera and Observability Paths

The active camera chain is separate from the ROS 2 control graph. The ZED GStreamer container produces front and rear RTSP streams; MediaMTX pulls and relays them as RTSP, HLS, or WebRTC; and the GUI embeds the WebRTC pages. At inspection time, the front path could be relayed, whereas the rear source returned a service error to MediaMTX.

Diagnostics are aggregated through ROS 2 on `/diagnostics_agg` and exposed to the GUI by the API. Foxglove provides a separate debugging path: the ROS 2 graph is made available through `savd_foxglove_bridge` to the browser-based Foxglove Studio. This path is intended for inspection and visualisation, not for normal vehicle control.

3.6 External Access Experiments

The deployed truck offers two conceptually different ways to obtain internal information from another machine:

1. call a known HTTP endpoint exposed by `savd_api`; or
2. run a ROS 2 client and discover or subscribe to the truck's ROS 2 graph directly through DDS.

The experiment compared basic reachability and discovery, not throughput or control performance. All API requests used read-only endpoints, and direct ROS 2 tests listed topics without publishing commands.

3.6.1 HTTP API Access

From a laptop connected to the truck through Tailscale, the following endpoint categories were tested:

```
GET http://<TRUCK_IP>:8000/modes/get_current_mode
GET http://<TRUCK_IP>:8000/vehicle/parameters
GET http://<TRUCK_IP>:8000/diagnostics
```

The API returned the current mode, vehicle parameters, and diagnostics. The client needed only normal IP connectivity and an HTTP implementation; ROS 2 and the SAVD custom message packages were not required. HTTP was also called successfully from inside the ROS 2 test container on the laptop. This confirmed that the container had IP connectivity to the truck even when ROS 2 discovery failed.

3.6.2 Direct ROS 2 Access

The direct test used the `ros:humble-ros-base-jammy` container image. Inspection of the truck containers did not reveal an explicitly configured `ROS_DOMAIN_ID`, so the tests used the ROS 2 default domain 0 and disabled localhost-only operation:

```
docker run --rm -it \
  -e ROS_DOMAIN_ID=0 \
  -e ROS_LOCALHOST_ONLY=0 \
  ros:humble-ros-base-jammy bash

source /opt/ros/humble/setup.bash
ros2 daemon stop
ros2 topic list -t --no-daemon
```

Three environments were examined. From Docker Desktop on macOS over Tailscale, the client discovered only its local `/parameter_events` and `/rosout` topics. A client container started on the truck discovered real SAVD topics, which provided a baseline that the image and ROS 2 commands were valid. Finally, a Linux machine on the same LAN as the truck discovered a substantial part of the truck graph, including `/ackermann_cmd`, `/diagnostics`, `/odom`, `/savd_sysmode/mode`, and the SAVD custom-message topics.

Table 6: Observed remote-access results.

Environment	Method	Observed result
macOS laptop over Tailscale	HTTP	Mode, parameters, and diagnostics were returned successfully
ROS 2 container on macOS over Tailscale	HTTP	API request succeeded, confirming ordinary IP connectivity
ROS 2 container on macOS over Tailscale	Direct ROS 2	Truck graph was not discovered; only local topics appeared
ROS 2 client running on the truck	Direct ROS 2	Real SAVD topics were discovered
Linux host on the same LAN	Direct ROS 2	Many truck topics were discovered successfully

The experiment indicates that direct ROS 2 access is possible in a suitable network environment but is more sensitive to host networking, multicast and discovery behaviour, Docker networking, VPN routing, domain configuration, and middleware settings. HTTP uses a known address and port and

therefore worked in the tested routed/VPN environment where DDS discovery did not. This result does not show that one protocol is universally superior; it shows that they expose different integration and deployment constraints.

Table 7: Practical differences observed between the two access approaches.

Aspect	HTTP API	Direct ROS 2
Client dependency	Any HTTP client	ROS 2 installation or container and compatible message definitions
Discovery and addressing	Known IP address and port	DDS discovery and matching ROS domain and network settings
Exposed scope	Endpoints explicitly implemented by the API	Potentially the wider ROS 2 graph
Data representation	JSON selected by the API	Native ROS 2 standard and custom message types
Tested remote behaviour	Worked over Tailscale	Did not discover the graph from macOS Docker over Tailscale
Tested local behaviour	Worked	Worked on the truck and from a Linux host on the same LAN
Access boundary	API can filter allowed operations	Requires separate controls to avoid exposing the complete graph

3.7 Current Issues and Limitations

The inspection captured a runtime snapshot, not a permanent judgement about the platform. [Table 8](#) lists the principal issues observed at that time.

Table 8: Runtime issues observed during the June 2026 inspection.

Component	Observation	Impact
Rear camera	MediaMTX could not pull the rear source; the source returned a service error	Rear video was unavailable through the normal GUI path
U-Blox GPS	The mounted launch file did not start the real GPS node; only static transforms were running	A healthy container did not imply that a live GPS fix existed
Jetson statistics	<code>savd_jetson_stats</code> exited with code 127 because of its runtime or socket setup	Host-statistics diagnostics were stale
Physical joystick	No <code>/dev/input/js*</code> device was present, while the health check could still pass	The physical joystick path could not be considered operational
Vehicle diagnostics	Some entries reported an error when no drive command had yet been received	A GUI “system error” did not necessarily mean loss of hardware connectivity
Camera diagnostics	ROS ZED diagnostics were stale while the deployed video path used direct GStreamer	Diagnostic state did not accurately represent the active media path

The source inspection also identified maintainability issues, including a service-timeout spelling mismatch, a ROS 2 message-type mismatch for a waypoint target subscription, inconsistent diagnostic keys, and a possible division by zero in a pure-pursuit geometry function. These findings should be validated and corrected independently; they were not modified during the read-only study.

Several API routes can change the physical state of the truck, including mode changes, joystick commands, waypoint goals, gear selection, differential lock, and fan settings. They were excluded from the remote-access experiment. Container restart and camera-recovery scripts were also excluded because they modify runtime state. Future tests involving actuation require a secured vehicle, an operator with immediate emergency-stop access, a known current mode, verified hardware connectivity, and a dedicated test procedure.

Four limitations constrain the interpretation of the results:

- The architecture description is a snapshot of the system deployed on 2–3 June 2026. Container versions, launch files, and runtime status may change.
- The remote-access experiment measured endpoint reachability and ROS 2 topic discovery only. It did not measure latency, jitter, throughput, packet loss, or long-duration reliability.
- The exact Docker network mode used by the same-LAN Linux client was not recorded in the original notes. Because DDS discovery is sensitive to bridge versus host networking, this should be captured in a future reproduction.
- No actuation command was sent. The study confirms software paths and read-only visibility, but it does not validate end-to-end physical motion or safety behaviour.

Despite these limitations, the analysis establishes the architectural facts needed by the rest of the project: ROS 2 remains the truck’s internal middleware; command arbitration must not be bypassed; custom interfaces and launch remappings matter for integration; the existing HTTP API provides a controlled remote boundary; and direct ROS 2 access depends strongly on the network environment.

4 Studied example : Eclipse SDV

[hyperref](#)

4.1 Why Eclipse SDV

The Eclipse Software Defined Vehicle (SDV) Working Group is a collaborative, open-source initiative hosted by the Eclipse Foundation. It unites automotive industry leaders, including automakers, suppliers, and technology companies, to build a common software foundation for the next generation of mobility. The ecosystem provides a vendor-neutral, code-first environment where organizations can co-develop modular, interoperable, and industry-ready software components for modern vehicle architectures.

The Eclipse SDV Working Group is especially relevant to this project because it provides open-source components that can be studied and deployed without access to a proprietary manufacturer platform. Its stated mission is to support a scalable, modular, extensible, industry-ready open vehicle software platform, and its landscape covers projects for in-vehicle and cloud-side use. By providing the essential “plumbing” for vehicle software—such as standardized data models, communication middleware, and orchestration tools—Eclipse SDV allows developers to focus on higher-level applications rather than reinventing basic hardware-access layers.

4.2 Eclipse Structure and main projects

The structure of Eclipse SDV is designed as an integration hub where platform-level automotive initiatives converge. The ecosystem spans the entire vehicle-edge-cloud continuum, providing tools and runtimes that facilitate modern development practices, continuous integration, and architectural alignment. Its projects act as modular building blocks that can be composed to fit various end-to-end architectures.

One of the central foundational initiatives within this ecosystem is the Eclipse Safe Open Vehicle Core (S-Core). S-Core aims to develop a secure, open-source core software stack tailored specifically for

Software-Defined Vehicles. It targets embedded, high-performance Electronic Control Units (ECUs) and focuses on ensuring multi-processor interoperability and functional safety. S-Core serves as the foundational operating environment upon which other vehicle services, diagnostics, and applications can securely execute.

Alongside S-Core, the Eclipse SDV landscape includes several other critical projects that handle specific layers of the vehicle architecture. Two key examples are:

- Eclipse Leda: A ready-to-use, Linux-based software distribution that bundles various open-source SDV components into a functional, deployable whole.
- Eclipse Kuksa: An essential data orchestration layer that acts as the "connective tissue" between hardware sensors and high-level applications, natively supporting the COVESA Vehicle Signal Specification (VSS).

Both Leda and Kuksa play a vital role in mapping and translating vehicle signals, and their specific functionalities will be detailed further in the following sections.

4.3 Eclipse Leda

Eclipse Leda is an open-source project that provides a functional, ready-to-use Linux-based software distribution tailored for Software-Defined Vehicles. Its primary purpose is to act as a "crystallization point" that brings together various independent SDV components—such as data brokers, update agents, and container runtimes—into a cohesive and working operating system image. Instead of requiring developers to manually compile, configure, and integrate complex middleware and foundational software from scratch, Leda provides a continuously available, pre-integrated baseline. Structure and Operation: Leda is structured as a highly modular, container-friendly operating system.

- Container Orchestration: It relies heavily on lightweight container runtimes (such as Eclipse Kanto) to manage and orchestrate isolated applications and services directly on the vehicle's edge computer.
- re-integrated Components: A standard Leda image comes bundled with essential SDV building blocks right out of the box. This typically includes the Eclipse Kuksa Databroker for data orchestration, MQTT brokers (like Eclipse Mosquitto) for communication, and vehicle update management agents.
- Deployment: Developers can deploy Leda in various environments, including virtualized cloud platforms, local QEMU emulators, or directly flashed onto physical embedded hardware (such as a Raspberry Pi or an automotive ECU).

By providing this unified environment, Leda allows developers to immediately begin writing and deploying vehicle applications, knowing that the underlying system architecture, networking, and security boundaries are already established.

4.4 Eclipse Kuksa

Eclipse Kuksa provides the essential data orchestration layer for the modern SDV ecosystem. It acts as the connective tissue between a vehicle's low-level hardware sensors (or actuators) and the high-level applications that require access to vehicle data.

In traditional vehicle architectures, applications must understand the specific, proprietary protocols of individual ECUs or CAN buses to read data. Kuksa eliminates this dependency by standardizing all vehicle data into a single, uniform format based on the COVESA Vehicle Signal Specification (VSS).

Structure and Protocols: The technical core of the project is the Kuksa Databroker, a highly efficient, centralized, in-vehicle server written in Rust.

- gRPC API: The Databroker exposes high-level, language-agnostic gRPC APIs (such as `kuksa.val.v2`). This allows applications written in Python, C++, Rust, or other languages to query signals, subscribe to updates, and send actuation commands over a standard network interface instead of dealing with native automotive buses.
- Data Providers (Feeders): Because the Databroker itself does not talk directly to hardware, it relies on "Providers" (such as a Kuksa CAN Provider). These providers subscribe to the vehicle's native network, map the raw signals to their corresponding VSS paths (e.g., translating a specific CAN frame into `Vehicle.Speed`), and push that data to the Databroker.
- Target vs. Current Values: For actuation, Kuksa conceptually separates a "Target Value" (what the application requests, e.g., setting a window position) from a "Current Value" (what the physical hardware is currently reporting).
- Centralized Architecture: Unlike decentralized publish/subscribe middleware, the Kuksa Databroker operates on a centralized client-server model, making it the single source of truth for the vehicle's state tree

4.5 CarSim Example

4.5.1 Original Example: VSS to VSS

The CarSim example is a demonstration project provided by the Eclipse ecosystem that illustrates how distinct software applications can communicate through standardized VSS data points using the Kuksa Databroker.

What it does and how it works: The simulation consists of two Python components that don't share direct link with each other, relying entirely on the Databroker for data exchange:

- Driversim (The Feeder): This component acts as an open-loop data generator. It simulates different driving behaviors ("Good driver" vs. "Bad driver") by generating data for the accelerator, brake, and steering angle without reading back the vehicle's actual state. A Python runner script bridges this simulator to Kuksa, continuously publishing target VSS data points like `Vehicle.Chassis.Brake.PedalPosition` to the Databroker via the Kuksa Python API.
- Carsim (The Simulator): This component runs a basic kinematic "bicycle vehicle model". Its runner script subscribes to the pedal and steering data points published by the driver. It uses these inputs to calculate new physical states (speed, lateral/longitudinal acceleration) and publishes the resulting values back to the Databroker under data points like `Vehicle.Speed` and `Vehicle.Acceleration.Lateral`.
- Finally the user can open a client cli interface to subscribe to the data sent by the CarSim engine and the DriverSim engine.

The example can be run locally using individual Docker containers for the Databroker, the Carsim, and the Driversim, all communicating over a shared container network, or it can be deployed directly onto a device running Eclipse Leda via Kanto container management (the chosen solution in our case).

Relevance and Limitations for Our Study Case: This example is useful for our study because it proves the core SDV concept: applications (like a driver input and a vehicle physics model) can be completely isolated and still function seamlessly together simply by agreeing on a VSS data dictionary and a central broker. It serves as a clean, best-case-scenario reference for how software components interact in a native VSS environment.

However, compared to the AVL robotic truck architecture, this example has significant limitations:

- Pure VSS Assumption: The Carsim example assumes the entire ecosystem natively speaks VSS. On our truck, the underlying communication relies entirely on ROS2 and DDS.
- Centralized vs. Decentralized: Carsim uses a single centralized Databroker for all signal routing. The truck uses a decentralized ROS2 graph where topics are published peer-to-peer.

- Lack of Arbitration: Driversim pushes inputs blindly. On the real truck, multiple command producers exist, and a dedicated `savd_syscontrol` node must arbitrate these commands based on the active mode state machine before anything reaches the wheels.
- Data Types: The VSS signals in Carsim are simple, discrete values (e.g., floats for speed). The truck relies heavily on complex, nested ROS2 message types (like `geometry_msgs/msg/TwistStamped` or custom SAVD interfaces) that have no direct primitive equivalent in VSS.

Because of these differences, we cannot simply copy the Carsim architecture; we must design a bridge that respects the truck’s existing ROS2 middleware while exposing the necessary data to a Kuksa-like environment.

4.5.2 Truck Adapted Example: ROS2 to VSS

To reflect more the architecture of the AVL robotic truck, we modified the VSS CarSim demonstration to introduce ROS2 as the internal communication middleware. In the original Eclipse configuration, the driver simulator acted as a direct Kuksa feeder, pushing target VSS data points like `Vehicle.Chassis.Brake.PedalPosition` directly to the Databroker via gRPC.

In the adapted architecture, the driver component (`driver.py`) was rewritten as a native ROS2 node (`DriverSimNode`). Instead of connecting to Kuksa, this node publishes standard ROS2 message types (`std_msgs/msg/UInt8` for the accelerator and brake, and `std_msgs/msg/Int16` for the steering angle) to local topics such as `/vehicle/chassis/accelerator` and `/vehicle/chassis/steering` at a fixed interval of 10 Hz. To support this execution environment, the container setup was updated from a basic Python image to a `ros:humble-ros-base` image (cf the modified Dockerfile).

The internal behavior and output of the driver remain identical. The kinematic algorithms calculating the simulated vehicle control states (`driver_base.py`) were untouched, meaning the exact same values are generated at the exact same simulated time steps. The simulation results don’t change. The structural change is only in the transport layer, moving from centralized gRPC communication to decentralized DDS publish/subscribe mechanisms, to mirror the data flow of the AVL truck

4.6 ROS2 & VSS : Bridge

This way of mapping data between ROS2 and VSS signal is the based for the ROS2-VSS bridge. The goal is to translate ROS2 signal in VSS to allow the user to deal with the information he wants through a databroker such as Kuksa, with API subscription and so on. A ROS2-to-VSS bridge is a translation layer that sits between the ROS2 graph and a VSS-based signal store (for example we used the [Eclipse Kuksa](#) Databroker). It’s not a protocol but an application-level component tailor made by the developers, because nothing can perform the translation for every ROS2 to VSS bridges as ROS2 structure is unique for each project. Architecture in practice: the Bridge Node is a normal ROS2 node (so it shares the DDS domain like any other participant) that also holds a client connection from the databroker. On the sensor path, it subscribes to ROS2 topics, pulls out the relevant fields, converts units/types, and pushes them to the Databroker as VSS current values. On the actuation path, it subscribes to VSS target values from the Databroker and republishes them as ROS2 topics, service calls, or action goals. A `mapping.yaml` declares which ROS2 field corresponds to which VSS path. This file is the actual translation table; the bridge node is just the engine that executes it.

4.7 Challenges

- **No semantic correspondence.** ROS2 message types are arbitrary and project-defined; VSS is a fixed catalog. Nothing inherently links `geometry_msgs/Twist.linear.x` to `Vehicle.Speed`. Every mapping has to be hand-declared, in the same way that the existing `kuksa-can-provider` requires an explicit DBC–VSS mapping file rather than inferring it.
- **Data model mismatch.** VSS leaves are scalar primitives with a fixed datatype, unit, and range. ROS2 messages are frequently nested, variable-length, or sensor-specific (point clouds,

images, occupancy grids) with no VSS equivalent. Only the subset that maps to discrete signals can therefore be bridged.

- **Rate and update semantics.** ROS2 topics can publish at hundreds of hertz (for example, IMU or LiDAR data), whereas VSS/gRPC streams are not designed for that volume. The bridge needs throttling, on-change filtering, or interval-based sampling.
- **No current/target distinction in ROS2.** VSS actuators separate a current value from a target value. ROS2 has no native equivalent, so the integration must decide whether a target write becomes a topic publication, a service call, or an action goal, and how the resulting current value is reported.
- **Unit and type safety.** ROS2 messages rarely carry units explicitly. VSS enforces datatype, unit, and range constraints per signal, so the bridge must validate, convert, and reject or clamp invalid values.
- **Dynamic graph versus static catalog.** ROS2 nodes and topics can appear and disappear at runtime, while the VSS tree is versioned and largely static. The mapping configuration must be maintained as the ROS2-side system evolves.
- **Transport paradigm mismatch.** DDS is decentralized and based on publish/subscribe communication, whereas the Kuksa Databroker follows a centralized client/server model. The bridge adds both a communication hop and a possible single point of failure.
- **Namespace mismatch in multi-robot or fleet contexts.** ROS2 namespaces can distinguish several robots, whereas one VSS tree normally represents one vehicle. A fleet integration must either define an additional mapping convention or operate one Databroker per vehicle.
- **Security model mismatch.** SROS2/DDS Security and Kuksa's token-based access control are independent systems. A production bridge must reconcile both trust models.

4.8 Existing Solutions

As the bridge is something unique for every case, no premade projects are existing to map every ROS2 architectures to VSS. However, some project proposed an idea of the architecture to use for the Bridge.

- **Kuksa's provider pattern.** Kuksa provides integrations for transports such as CAN/DBC, SOME/IP, and Zenoh. They subscribe to a native transport, apply a declarative mapping, and forward the resulting signals to the Databroker. A ROS2 bridge can follow the same pattern using a Kuksa client SDK inside a ROS2 node.
- **ros_gz_bridge as an architectural reference.** Gazebo's ROS2 bridge is not VSS-specific, but its YAML configuration maps ROS topic names and types to Gazebo equivalents. This structure is close to the proposed ROS2-VSS mapping approach.
- **Generic VSS middleware libraries.** Libraries such as `vss-lib` encapsulate access to a Kuksa Databroker. Such a library can be embedded in a custom ROS2 node instead of implementing the complete gRPC client layer directly.
- **Commercial vehicle-data platforms.** Vehicle-data platforms can map simulation and vehicle signals to VSS and integrate them with ROS2-based systems such as Autoware. These examples demonstrate VSS integration at system level rather than through a universal topic-by-topic bridge.
- **rmw_zenoh.** ROS2 can use Zenoh as a middleware implementation, while Kuksa also provides a Zenoh provider. This may reduce the amount of transport-specific code, although an explicit mapping between ROS2 topics and VSS signals is still necessary.

In practice, the bridge remains project-specific. The Kuksa provider pattern supplies the execution model, while a file such as `mapping.yaml` declares the correspondence between ROS2 message fields and VSS paths.

4.9 ROS2 to VSS Bridge for AVL Truck

Because the Databroker expects only standardized VSS signals via gRPC and the modified driver simulator now outputs arbitrary ROS2 messages, a translation layer is required to connect the systems. The `ros2_vss_bridge.py` script serves this exact purpose. It functions as a hybrid component: a native ROS2 node that subscribes to the driver’s command topics, coupled with a Kuksa client connected to the Databroker through an active gRPC channel.

A critical aspect of implementing this bridge is understanding how and when translation occurs. The semantic mapping—associating the ROS2 topic `/vehicle/chassis/accelerator` with the VSS path `Vehicle.Chassis.Accelerator.PedalPosition`—is defined statically exactly once within the bridge’s code. However, the actual payload translation must be executed dynamically for every single packet. Every time the ROS2 node receives a message callback, it must actively extract the payload (`msg.data`), cast it into a Kuksa Datapoint, and transmit it over the network to the Databroker to update the vehicle’s state tree.

This per-packet translation is computationally viable for discrete scalar values (like pedal positions updating at 10 Hz), but it exposes a fundamental limitation of VSS when applied to high-bandwidth or complex data types. If the bridge were tasked with handling the truck’s ZED camera video feed, it would be impossible to translate every individual video frame into a VSS structure. VSS and the Databroker are designed for state representation, not for transporting high-frequency binary blobs or real-time media streams.

For such high-bandwidth data in a ROS2-to-VSS environment, the system translates the metadata, not the data itself. The bridge would register the state of the stream (e.g., publishing a VSS string containing the active RTSP URL, or a boolean indicating `Camera.IsActive`) during initialization or when the state changes. The actual video frames would bypass the VSS bridge entirely, streaming out-of-band via dedicated media pipelines like GStreamer or MediaMTX. This confirms that the bridge’s role is to maintain semantic interoperability of the vehicle’s state, rather than acting as a universal tunnel for all internal data.

5 Conclusion

This project allowed us to dive into the world of the Software-Defined Vehicle (SDV) and to fundamentally understand both its practical usage and the main challenges it presents. Throughout our research, we discovered the absolute necessity of open-source SDV standards to connect vehicles together, regardless of the manufacturer or the specific internal systems they choose to implement.

Furthermore, we closely studied the architecture of the AVL robotic truck to understand how it—and modern vehicles in general—can be built. This hands-on analysis gave us a solid and accessible understanding of what actually goes on inside an SDV’s software stack, from containerized deployments to complex middleware routing.

Finally, we successfully implemented and ran basic examples, such as the ROS 2-to-VSS bridge. This practical step allowed us to directly confront basic SDV solutions and experience firsthand the real-world challenges, such as semantic data translation, that inevitably occur while developing software-defined vehicles.

References

- [1] Eclipse SDV Working Group, “What is a Software-Defined Vehicle?,” Eclipse Foundation, 2026. <https://sdv.eclipse.org/>

- [2] Eclipse SDV Working Group, “Eclipse Software-Defined Vehicle Working Group Charter,” Eclipse Foundation.
<https://sdv.eclipse.org/about/charter/>
- [3] AUTOSAR (AUTomotive Open System ARchitecture), “Explanation of Adaptive Platform Design and Architecture,” Release Overview.
<https://www.autosar.org/standards/adaptive-platform>
- [4] United Nations Economic Commission for Europe (UNECE), “UN Regulation No. 156 - Software Update and Software Update Management Systems (SUMS),” Transport Guidelines.
<https://unece.org/transport/vehicle-regulations>
- [5] Connected Vehicle Systems Alliance (COVESA), “Vehicle Signal Specification (VSS),” official data model schema and repository.
https://github.com/COVESA/vehicle_signal_specification
- [6] SOAFEE Special Interest Group, “Scalable Open Architecture for Embedded Edge: Architectural Overview and Reference Implementation,” 2022–2026.
<https://www.soafee.io/>
- [7] International Organization for Standardization, “ISO 26262: Road Vehicles — Functional Safety,” Standard Series, 2018.
<https://www.iso.org/standard/68383.html>
- [8] United Nations Economic Commission for Europe (UNECE), “UN Regulation No. 155 - Cyber Security and Cyber Security Management Systems (CSMS),” Transport Guidelines.
<https://unece.org/transport/vehicle-regulations>
- [9] International Organization for Standardization / SAE International, “ISO/SAE 21434: Road Vehicles — Cybersecurity Engineering,” 2021.
<https://www.iso.org/standard/70918.html>
- [10] Connected Vehicle Systems Alliance, “About the COVESA Alliance and Connected-Vehicle Specifications,” 2026.
<https://www.covesa.global/>
- [11] Eclipse Foundation, “Eclipse SDV Project Landscape and Open Source Ecosystem Portal.”
<https://landscape.sdv.eclipse.org/>
- [12] digital.auto, “Vehicle Signal Specification (VSS) Tree Explorer, Prototyping, and Cloud Deployment Dashboard.”
<https://vss.digital.auto/>
- [13] Eclipse Leda Project, “Eclipse Leda: Vehicle application examples and use cases,” GitHub Repository.
<https://github.com/eclipse-leda>
- [14] Eclipse KUKSA Project, “KUKSA Databroker: A modern in-vehicle VSS server written in RUST,” GitHub Repository.
<https://github.com/eclipse-kuksa/kuksa-databroker>
- [15] Eclipse Kanto Project, “Eclipse Kanto: Container Management and Software Updates for Edge Devices,” GitHub Repository.
<https://github.com/eclipse-kanto>
- [16] Eclipse Zenoh Project, “Eclipse Zenoh: Zero Overhead Pub/Sub, Store/Query and Compute,” GitHub Repository.
<https://github.com/eclipse-zenoh/zenoh>

- [17] ROS 2 Project, “rmw_zenoh: RMW for ROS 2 using Zenoh as the middleware,” GitHub Repository.
https://github.com/ros2/rmw_zenoh
- [18] Gazebo Simulator Project, “ros_gz_bridge: Bridge communication between ROS and Gazebo Transport,” GitHub Repository.
https://github.com/gazebo-sim/ros_gz
- [19] Connected Vehicle Systems Alliance (COVESA), “Vehicle Signal Specification - standardized way to describe automotive data,” GitHub Repository.
https://github.com/COVESA/vehicle_signal_specification